To retain the inheritance semantics, the base class attributes are going to be merged with the subclass ones, so the associated subclass entity table contains both. This is similar to the table-per-class inheritance strategy, with the distinction that the base class is not mapped to a database table (hence, it cannot be used in polymorphic queries or associations).

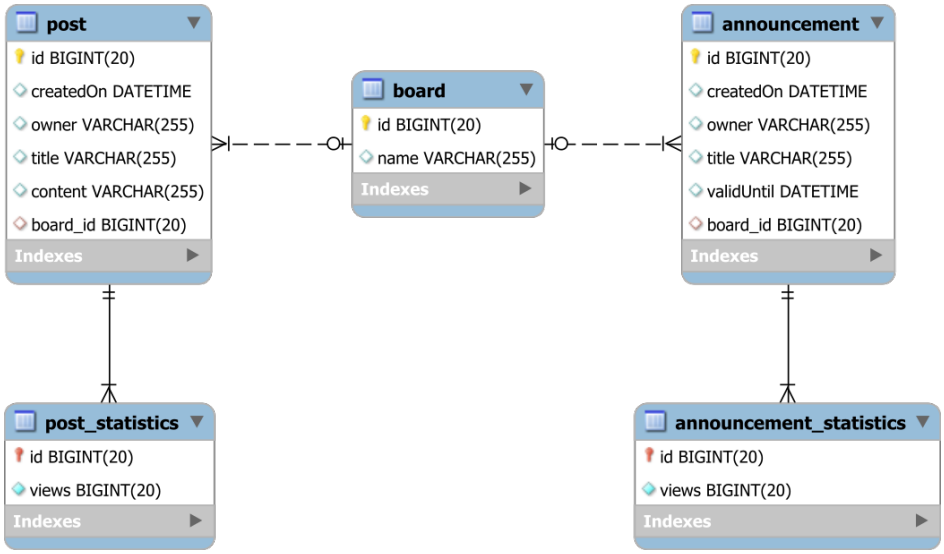

Figure 11.6: Mapped superclass

The `post` and the `announcement` tables feature columns that were inherited from the `Topic` base class. When persisting the same `Post` and `Announcement` entities and saving statistics using the newly defined entity classes:

```
TopicStatistics postStatistics = new PostStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);

TopicStatistics announcementStatistics =
    new AnnouncementStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Hibernate generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id )
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Inheritance',
    'Best practices', 1)
```

```sql
INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Release x.y.z.Final',
    '2016-02-17 09:11:07.529', 2)

INSERT INTO post_statistics (views, id) VALUES (1, 1)

INSERT INTO announcement_statistics (views, id) VALUES (1, 2)
```

Querying for statistics require specifying the actual `Topic` subclass statistics:

```java
PostStatistics statistics = entityManager.createQuery(
    "select s from PostStatistics s join fetch s.topic t where t.id = :postId",
        PostStatistics.class)
.setParameter("postId", postId)
.getSingleResult();
```

This entity query generates a single SQL join statement:

```sql
SELECT
    s.id AS id1_4_0_,
    p.id AS id1_3_1_,
    s.views AS views2_4_0_,
    p.board_id AS board_id6_3_1_,
    p.createdOn AS createdO2_3_1_,
    p.owner AS owner3_3_1_,
    p.title AS title4_3_1_,
    p.content AS content5_3_1_
FROM post_statistics s
INNER JOIN post p ON s.id = p.id
WHERE p.id = 1
```

Polymorphic queries against the `Topic` class are not permitted and the `Board` entity cannot define a `@OneToMany Topic` collection either.

Performance considerations

Although polymorphic queries and associations are no longer permitted, the `@MappedSuperclass` yields very efficient read and write operations. Like single and table-per-class inheritance, write operations require a single insert statement and only one table is used for reading data.

Inheritance best practices

All the inheritance mapping models above require trading something in order to accommodate the impedance mismatch between the relational database system and the object-oriented Domain Model.

The default single table inheritance performs the best in terms of reading and writing data, but it forces the application developer to overcome the column nullability limitation. This strategy is useful when the database can provide support for trigger procedures, and the number of subclasses is relatively small.

The join table is worth considering when the number of subclasses is higher, and the data access layer does not require polymorphic queries. When the number of subclass tables is large, polymorphic queries require many joins, and fetching such a result set has a noticeable impact on application performance. This issue can be mitigated by restricting the result set (e.g. pagination), but that only applies to queries and not to `@OneToMany` or `@ManyToMany` associations. On the other hand, polymorphic `@ManyToOne` and `@OneToOne` associations are fine since, in spite of joining multiple tables, the result set can have at most one record only.

Table-per-class is the least effective when it comes to polymorphic queries or associations. If each subclass is stored in a separate database table, the `@MappedSuperclass` Domain Model inheritance might often be a better alternative.

Although a powerful concept, Domain Model inheritance should be used sparingly and only when the benefits supersede trade-offs.

12. Flushing

As explained in the write-based optimizations section, the Persistence Context acts as a transactional write-behind cache. The Hibernate `Session` is commonly referred to as the first-level cache since every managed entity is stored in a `Map`, and, once an entity is loaded, any successive request serves it from the cache, therefore avoiding a database roundtrip.

However, aside from caching entities, the Persistence Context acts as an entity state transition buffer. Both the `EntityManager` and the Hibernate `Session` expose various entity state management methods:

- The `persist` method takes a transient entity and makes it managed.
- The `merge` method copies the internal state of a detached entity onto a freshly loaded entity instance.
- Hibernate also supports reattaching entity instances (e.g. `update`, `saveOrUpdate` or `lock`) which, unlike `merge`, does not require fetching a new entity reference copy.
- To remove an entity, the `EntityManager` defines the `remove` method, while Hibernate offers a `delete` method.

Like any write-behind cache, the Persistence Context requires flushing in order to synchronize the in-memory persistent state with the underlying database. At flush time, Hibernate can detect if a managed entity has changed since it was loaded and trigger a table row update. This process is called dirty checking, and it greatly simplifies data access layer operations.

So, when using JPA, the application developer can focus on entity state changes, and the Persistence Context takes care of the underlying DML statements. This way, the data access layer logic is expressed using Domain Model state transitions, rather than through insert, update, or delete SQL statements.

This approach is very convenient for several reasons:

- Entity state changes being buffered, the Persistence Context can delay their execution, therefore minimizing the row-level lock interval, associated with every database write operation.
- Being executed at once, the Persistence Context can use JDBC batch updates to avoid executing each statement in a separate database roundtrip.

However, having an intermediate write-behind cache is not without challenges and the Persistence Context can be subject to data inconsistencies. Since efficiency is meaningless if effectiveness is being compromised, this chapter aims to analyze the inner-workings of the flushing mechanism, so the application developer knows how to optimize it without affecting data consistency.

12.1 Flush modes

The Persistence Context can be flushed either manually or automatically.

Both `EntityManager` and the Hibernate native `Session` interface define the `flush()` method for triggering the synchronization between the in-memory Domain Model and the underlying database structures. Even so, without an automatic flushing mechanism, the application developer would have to remember to flush prior to running a query or before a transaction commit.

Triggering a flush before executing a query guarantees that in-memory changes are visible to the currently executing query, therefore preventing *read-your-own-writes* consistency issues.

Flushing the Persistence Context right before a transaction commit ensures that in-memory changes are durable. Without this synchronization, the pending entity state transitions would be lost once the current Persistence Context is closed.

For this purpose, JPA and Hibernate define automatic flush modes which, from a data access operation perspective, are more convenient than the manual flushing alternative.

JPA defines two automatic flush mode types:

- `FlushModeType.AUTO` is the default mode and triggers a flush before every query (JPQL or native SQL query) execution and prior to committing a transaction.
- `FlushModeType.COMMIT` only triggers a flush before a transaction commit.

Hibernate defines four flush modes:

- `FlushMode.AUTO` is the default Hibernate API flushing mechanism, and, while it flushes the Persistence Context on every transaction commit, it does not necessarily trigger a flush before every query execution.
- `FlushMode.ALWAYS` flushes the Persistence Context prior to every query (HQL or native SQL query) and before a transaction commit.
- `FlushMode.COMMIT` triggers a Persistence Context flush only when committing the currently running transaction.
- `FlushMode.MANUAL` disables the automatic flush mode, and the Persistence Context can only be flushed manually.

While the `FlushModeType.COMMIT` and `FlushMode.COMMIT` are equivalent, the JPA `FlushModeType.AUTO` is closer to `FlushMode.ALWAYS` than to the Hibernate `FlushModeType.AUTO` (unlike `FlushMode.ALWAYS`, `FlushModeType.AUTO` does not trigger a flush on every executing query).